Exploration of CNN Feature Transfer Across Real and Synthetic Domains

Matthew Thomas, Jacky Lin
COE 379L
Fall 2025
December 12, 2025

Introduction

One of the most consequential features of deep learning models is that they require an extensive amount of training data to be able to learn patterns and perform accurate inference. The abundance of data via digital technology, sensors, and the internet in the 21st century has allowed for the development of highly accurate machine learning models. However, as technologists try to bring deep learning to other domains, this can be a major roadblock to creating models that make predictions on images that are highly limited in nature. Some examples of this include AI models for use on radar images of enemy military vehicles and rare disease endoscopic images. This is the motivating factor behind synthetic data generation, which involves artificially creating images that deep learning models can train on to better learn the features of the target of interest when obtaining real images is difficult. For this project, we aim to explore how well the feature representations learned by a convolutional neural network (CNN) from images synthesized by generative AI models transfer to real-world photographic data and vice versa. Our motivating question: Can pretrained machine learning models be used to generate synthetic data that substitutes for real data? Synthetic images possess many differences from real images stemming from the prompt used to generate them, hallucinations, random factors, and more. This work aims to quantify the resulting performance gap in classification.

Data Sources & Technologies

The real-world data used for this project will be a subset of CIFAR-10, which is a widely known dataset containing a variety of differently labeled 32x32 pixel images. We decided to use this dataset because it is one of the standard datasets that are used to generate baseline/benchmark performances in computer vision tasks, and it is easily available via the Keras API. The subset we used contains only images of the animal classes (bird, deer, dog, cat, horse, frog), but with the full 6,000 images of each class.

For the synthetic dataset, images were generated through a StabilityAI text-to-image diffusion model from Hugging Face, called “sd-turbo”. This specific model was chosen because of its speed, as it can create an acceptable animal image from a text prompt in a single network evaluation on a CPU. The creation of a single image took around three seconds, making it the optimal model due to our time and resource constraints. By prompting the diffusion model with “A(n) {animal} of any species in a simple natural setting similar to CIFAR-10 style," 3,000 images of each animal class were generated. A target of 3,000 images per class was chosen due to computational and time constraints. The images generated by the model were by no means perfect, as some of the animals were incorrectly shaped or created with missing or too many eyes/limbs, among other imperfections. Also, most of the images had the same point of view of the animal. A sample synthetic frog and horse image is shown below.

This project was developed in Python with Jupyter Notebook, which provided a flexible environment for data preprocessing, model construction, and evaluation. TensorFlow and its high-level Keras API were used to design and train the CNN.

Finally, this project also deploys an inference server. This server uses the Flask API framework and is containerized using Docker. Using a Docker Compose setup, this project ensures simple and standardized usage of the server.

Methods

After the real and synthetic data were gathered, three experiments were conducted.

1. Train CNN on CIFAR-10, test on synthetic data
2. Train CNN on synthetic data, test on CIFAR-10
3. Train and test CNN on a mixture of the two data types

Each experiment utilized the same CNN architecture and assessed each model's performance by computing test accuracy, which is a reliable metric since we are working with a balanced dataset. This allows for numerical understanding of how well the model trained on synthetic data can generalize to real data and vice versa.

Data Preparation

Because the diffusion model outputs a 512x512 pixel colored image, it was necessary to resize these synthetic images to 32x32 pixels in order to make them compatible with CIFAR-10 throughout the machine learning process. This was achieved with Keras' load_img() function. As expected, the quality and resolution of the synthetic images decreased drastically after the downsampling. While this may sound concerning, it is important to remember that the CIFAR-10 images are purposely low-resolution, so the synthetic images we use must match that. Some images were harder to discern than others, most likely due to the diffusion model having variation in its success in producing accurate animal images.

Conveniently, CIFAR-10 is one of the pre-loaded datasets that can be imported into a Python environment via the Keras API. We took advantage of this to obtain the full CIFAR-10 dataset before removing all of the non-animal images. The synthetic images were converted into an array, and all images were normalized by dividing by 255, the maximum pixel value. This ensures that the data is numeric and each feature contributes proportionally to the training process, respectively. From there, the data was split into train and test sets depending on the experiment, and the class label was one-hot encoded using Keras' to_categorical() function.

A synthetic bird image before and after downsampling and normalization.

Model Architecture & Training

For this project, we utilized an alternate LeNet-5 CNN architecture. During the length of this project, we also experimented with architectures optimized for the CIFAR-10 data set. However, the resulting performances were about the same as those of LeNet-5, reaching about 70% to 80% accuracy on the test and training sets for real data. Consequently, we decided to continue using the LeNet-5 CNN architecture due to our familiarity with the architecture.

When training each model, early stopping was utilized to save unnecessary training time and prevent overfitting. Specifically, training would conclude if validation loss (or accuracy in some cases) did not improve over 10 consecutive epochs.

Experiment Details & Results

Real Trained Model

We started our experiments by training and testing a CNN on the CIFAR-10 dataset to obtain a baseline performance that could be achieved without any synthetic data. We used a 75:25 train-test split, and the test accuracy achieved was 74%. This indicates that the model was

able to learn some useful features, but still has issues with generalization. This is likely due to the poor image quality of the images overall, due to their small size. This provides less detail, texture, and edge information for the model to extract, likely making it harder to correctly distinguish the categories. It is also worth noting that while we did not directly test a model on synthetic data, the validation accuracies during training on synthetic data were 99%, implying that prediction on synthetic images after training on the same kind is an "easy" task.

Although it does not have much real-world utility, our first experiment was to train a model on CIFAR-10 and test it on the synthetic dataset to analyze domain adaptation. The model weights were the same as those used to achieve the baseline performance, so all we did was test that model that was trained on CIFAR-10 on the full synthetic dataset. The model achieved a test accuracy of 53%. This result is unsurprising because the synthetic images are fundamentally different from the real CIFAR-10 images. Even though both datasets contain pictures of animals of the same class, the synthetic images showed different color distributions, textures, lighting patterns, and other variances. This model was only trained on real data; consequently, it learned the features specific to the real images and, as a result, could not generalize to the synthetic images. Overall, this experiment shows that synthetic images do not generalize well to models trained on real data, which is acceptable because there would not be a scenario where achieving good performance on synthetic data is preferred over real data.

Synthetic Trained Model

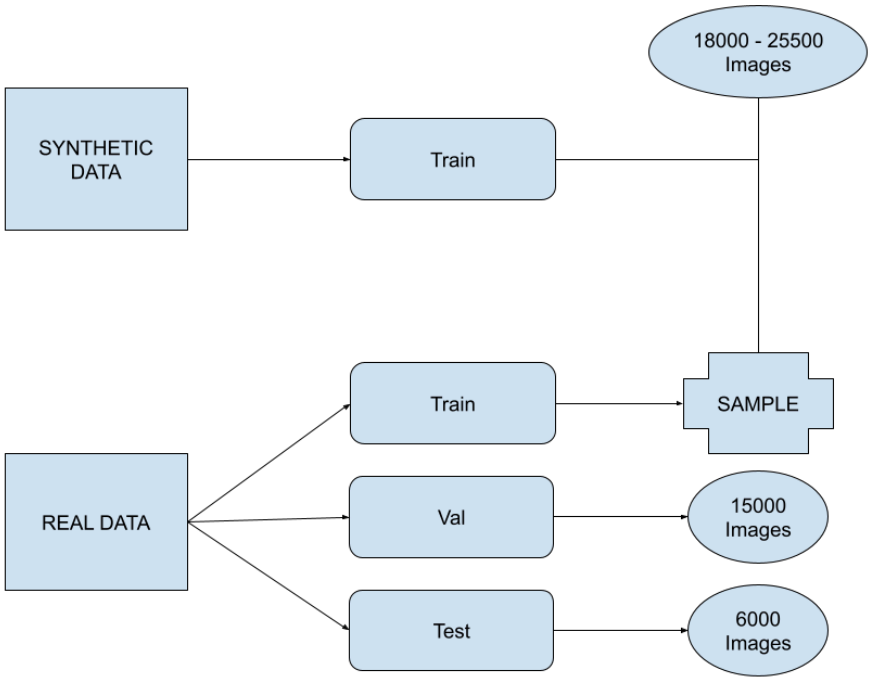
Next, with the most real-world utility, we trained a model on the entire synthetic dataset and tested it on CIFAR-10. The test set used was the natural test split that arises when you import CIFAR-10 via the Keras API, which includes 1000 images for each class. This test set was kept isolated from the machine learning process and was used to provide an unbiased assessment of performance on any model we tested on CIFAR-10. The test accuracy achieved for this experiment was a low 28%. Reasons for this are potentially similar to reasons why the first experiment achieved low accuracy. However, given that this test accuracy was half the previous, we suspect that domain adaptation may be easier going from real to synthetic than the other way around, highlighting that this is a directional problem.

Mixture Trained Model

Lastly, the bulk of our experiments included analyzing how varying the amount of real data added to the synthetic training set affected test accuracy on CIFAR-10. To do this, we built an entirely new workflow. First, we separated the data into two training sets: synthetic and real, a validation set, and a test set. We built the validation set as a separate set of real data equal to half of the natural training split that the Keras API provides for CIFAR-10, which comes out to 25000 images. The other half would become the real training set, being used as a bank of additive data

that could be combined with the synthetic training set for our experiments. Because the split was random, the classes were no longer perfectly balanced in the validation and real training sets. The test set was the same data as before.

We ran experiments varying the fraction of the real training data that would be added to the synthetic training data. The specified fraction of real data was randomly sampled. For example, for the 0.2 experiment, a random sample of 20% of the additive data set was added to the training set. The CNN was then trained on all of the synthetic data plus those additional real images, and then tested on the test set. Due to the random sampling, the training sets no longer had a perfect class balance, but the imbalance was minimal. A diagram to visualize the split is shown below.



The models were trained to optimize validation accuracy. We attempted to increase the fraction of real data until we achieved the baseline test accuracy (74%), but as seen in the results table below, that number was never achieved.

Fraction of Real Data Used	Real Training Images	Real-to-Synthetic Ratio	Train Accuracy	Test Accuracy
0	0	0	0.9954	0.2777
0.1	1500	0.077	0.9844	0.4962
0.2	3000	0.143	0.9739	0.5543
0.3	4500	0.2	0.9750	0.6113
0.4	6000	0.25	0.9663	0.6187
0.5	7500	0.294	0.9703	0.6400
1.0	15000	0.455	0.9673	0.6605

The results were as expected; as the amount of real data in the training set increases, the model performs better on the test set of solely real images. This is reasonable because if the model can learn from more real images, it can generalize that knowledge to other real images. The largest jump in test accuracy came from adding 10% of the real data during training, insinuating that just the mere presence of real images in the training set has a large impact. It is interesting to note that train accuracy fell at a much lower rate than the increase in test accuracy, but that may be due to the heavy imbalance between real and synthetic data within the training sets. The test accuracy never matched the accuracy that was achieved when training and testing on real images, indicating that for this problem, you need a lot more than ~45% real images in the training set to get baseline accuracy.

Discussion

There were several choices that we made throughout the experiments that require additional context. For instance, the rationale for varying the fraction of real data added to the training set over directly varying the ratio of real to synthetic data that we would train on is that in the real-world case of this problem, researchers most likely have a set of real data that is limited and highly coveted. Training a model with the least amount of this data becomes the goal, so it would make more sense to vary the amount used rather than adjust the real-to-synthetic proportion, which can simply be achieved by changing the amount of synthetic data. The latter is unfavorable because you would want to train with the most synthetic data that you can.

Second, it is known that training text-to-image diffusion models requires an enormous amount of labeled image data to be able to generate accurate images from text descriptions. So, getting an AI model to produce an image of something that has limited images would be nearly

impossible, since it could not be well represented in the training set. In short, a diffusion model could not be used to create additional data that is limited in nature, i.e., a model cannot produce a radar image of enemy aircraft if it was not trained on a plethora of similar images. Our experiments were possible because the sd-turbo model was clearly trained with labeled animal images. While we understand that using diffusion models cannot be used to solve the scarce data problem, this work works well as a proof of concept to show that domain adaptation is difficult for CNNs. In a future where diffusion models may be able to accurately generate images dissimilar to those on which they were trained, this work is important to show that CNNs still struggle to generalize between synthetic and real images.

References

[1] Krizhevsky, A. (2009). CIFAR-10 and CIFAR-100 datasets. Toronto.edu. https://www.cs.toronto.edu/~kriz/cifar.html

[2] stabilityai/sd-turbo · Hugging Face. (n.d.). https://huggingface.co/stabilityai/sd-turbo

[3] Team, K. (n.d.). *Keras Documentation: CIFAR10 small images classification dataset*. https://keras.io/api/datasets/cifar10/